

Improving Parallel Scalability in Web Server Architectures

Introduction

Web servers allow most websites to function according to the web files, for small government sites to large e-commerce platforms. Open source web servers like Apache or Nginx are often function for local governments, educational institutions, and NGOs for operating their web presence. As these platforms are increasing and the number of viewers are dependent on these in the same accessible time, it can cause parallel operations along with memory access.

This research discusses optimizing thread management, server response and memory use dependent on parallelism to improve web server performance. This research is intended to determine the main drawbacks in web server designs and to offer effective methods for managing increased traffic without sacrificing performance (Hennessy & Patterson, 2017).

Previous research has shown that using effective memory management and parallel processing models can considerably reduce contention and enhance server scalability, which is crucial for systems that handle a lot of traffic (Anderson & Velásquez, 2016).

Background / Related Work

Apache and Nginx are widely used web servers for managing incoming queries on websites. A process/thread-per-request architecture is applied for these web servers and for new incoming requests a new thread or process is created. This methodology is workful for low traffic performance but when there is a large number of threads constantly working and accessing the memory location, it creates performance challenges. The struggle for resource use will naturally cause delays for running tasks, resulting in larger latency and a reduced performance rate for the entire system. This becomes a serious issue for any site dealing with a considerable amount of traffic. It has been observed in past research studies that problem handling in the management of threads, along with locking issues, result in considerable delays in the handling of requests (Hennessy & Patterson, 2017).

To address all of these issues, we need to optimize thread interaction with memory and come up with better thread management algorithms.

Methodology / System Design

This research is to upgrade the way the Apache web server responds to incoming thread requests. Apache, like other web servers, handles new incoming connections by creating new threads or processes. But this method isn't very adaptable when the number of requests increases since the threads depend on shared memory for storing process data and cache, which can quickly become an obstacle. To address this, a lock-free queue is recommended as an option to deal with new requests. A lock-free queue permits threads to manage requests on its own, removing the requirement for locks. This approach minimizes conflicts, which improves performance throughout the process.

Furthermore, we recommend using per-thread memory queues. All of these would be used to store process data and cache it internally for each thread. This method would minimize the requirement for shared memory, thus reducing contention.

For asynchronous tasks, I/O batching or non-blocking I/O operations such as epoll or kqueue, if it is occupied, using these can make the system quicker and more flexible.

Assumptions & Performance Estimate:

Assumption	Quantitative Value	Evidence / Source
Concurrency Level	1000 concurrent users	Anticipated server capacity for managing high-traffic situations
Lock-Free Queue Efficiency	40% reduction in thread contention	Derived from investigations into lock-free data structures within high-concurrency settings (Anderson & Velásquez, 2016)
Response Time Improvement	250ms to <100ms	Projected latency reduction, contingent upon diminished contention and enhanced memory access efficiency (Hennessy & Patterson, 2017)
Energy Efficiency	30% reduction in energy consumption	Decreased power consumption, attributable to optimized thread management and minimized locking (Micron, 2017)
System Load Handling	Efficient under 1000+ concurrent users	Performance benchmark for thread management and I/O optimizations (Linux Documentation for epoll)

When lock-free queues and per-thread memory management are in setup, it can be expected that the system will operate much faster. It aims to decrease the typical response time reduced from 250ms to less than 100ms however it still requires to have the capacity to help 1000 people all at once. It is expected that such a shift will occur as there will be fewer threads struggling in memory as well as quicker access to memory.

Discussion

The research approach has become an effective method of getting around the issues associated with Apache's process/thread-per-request technique. Providing a lock-free queue and per-thread memory queues minimizes the requirement for shared memory and removes a lot of the disruptions which locking creates. As an outcome, Apache will be ready to cope with more individuals at a time and become more effective. Lock-free data structures allow high-concurrency configurations to run quickly while causing a lot less thread contention, according to research (Anderson & Velásquez, 2016).

The method does improve performance substantially, yet it has some drawbacks. The most difficult task involves setting up per-thread memory queues, meaning developers have to deal with memory for each thread individually. The result makes the system more complex and could indicate that the web server requires to be configured differently. However, studies have shown that this additional effort might be worth it because decreasing memory contention leads to big performance gains (Hennessy & Patterson, 2017). Furthermore, I/O batching helps things up by reducing down on the process of moving between contexts, but this may cause the process to take much longer when quick responses are required, like when dealing with interactive or real-time requests (NGINX, Inc., 2020).

When it comes to the environment, increasing the speed of a web server means using less energy. The system will consume less power when memory conflicts remain to a minimum and thread control is changed. Recent studies indicate that energy-efficient server designs that reduce unnecessary memory access and locking methods may assist data centers last longer and cost less to run (Micron Technology, 2017). By utilizing less energy, operating expenses go down but the web's infrastructure turns environmentally friendly as well. The changes are in compliance with attempts to make computer systems less harmful to the environment as demand for sustainable computers expands.

Conclusion

The research conducted suggested an approach to improve the Apache web servers more adaptable by making greater use of memory management and thread handling. Incorporating a lock-free queue, per-thread memory queues, and I/O batching methods is designed to make the system more efficient and less sensitive to thread contention. Such changes will make web sites more smoothly functioning in the future, it will be beneficial for the nature by using less electricity and these are also more effective. Moreover,

research should investigate other improvements such as using dynamic memory allocation and smart load balancing techniques to make web server architectures more adaptive and efficient

References

1. **Hennessy, J., & Patterson, D.** *Computer organization and design: The RISC-V edition* (6th ed.). Morgan Kaufmann Publishers.2017
2. **NGINX, Inc..** *NGINX documentation: Fine-tuning NGINX for peak performance*. 2020
3. **Jemalloc Contributors.** *jemalloc: A malloc implementation designed for wide use*.
4. **Linux Foundation.** *epoll(7) - Linux documentation page*.
5. **Apple Developer Documentation.** *kqueue(2) - Apple Developer Documentation*.
6. **Zhou, Y., & Guo, J.** Improving the scalability of multi-threaded servers requires memory management that considers parallelism. *International Journal of Cloud Computing and Services Science*. 2018
7. **Anderson, D., & Velásquez, S.** Improving web server performance using concurrency models: A study of how lock-free data structures and memory queues affect performance. *Journal of Web Engineering*, 2016